GENOX: Exploiting Thread Signals for Dynamic CPU Scaling in Genomics Workflows

Martin L. Putra*, In Kee Kim†, Haryadi S. Gunawi*, and Robert L. Grossman*
*Department of Computer Science, University of Chicago, {martinluttap, haryadi, rgrossman1}@uchicago.edu
†School of Computing, University of Georgia, inkee.kim@uga.edu

***Abstract*—DAG-structured genomics workflows process gigabytes to terabytes of sequencing data in shared clusters, where efficient CPU allocation is a critical resource management problem. However, existing CPU allocation approaches are insufficient for genomic workflows due to their unique challenges: input-dependent performance variability, contention-induced interference, and dynamic workflow branching.**

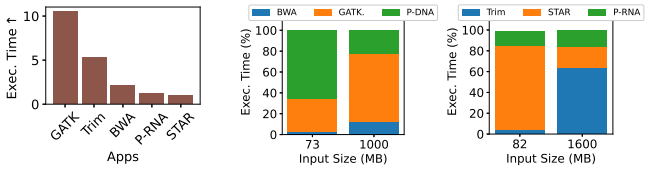
In this work, we first characterize production DNA and RNA sequencing workflows, identifying three critical characteristics: inflatable parallelism (thread surges up to 96×), stall heterogeneity (up to 2× slowdowns), and millisecond-scale idle bursts that make utilization often unreliable. These findings confirm that effective CPU management should satisfy three requirements: 1) responsiveness to thread bursts, 2) robustness under contention, and 3) adaptiveness to workflow dynamics. Motivated by our characterization study, we developed GENOX, an automated CPU quota management system that leverages thread counts as its primary signal for CPU scaling, rather than traditional metrics like utilization. Operating at 5 ms intervals through standard Linux cgroups, GENOX responds immediately to CPU demand phase changes while remaining stable under contention. Evaluation results show that GENOX outperforms state-of-the-art baselines, reducing application execution time by nearly 50%, eliminating straggler slowdowns of up to 200%, and improving end-to-end workflow completion time by over 20%.

***Index Terms*—Genomics workflows, dynamic CPU allocation, resource management, resource contention, scientific workflows**

I. INTRODUCTION

Genomics workflows consist of a set of bioinformatics applications organized into DAG-structured pipelines that process gigabytes to terabytes of genomic and biological data while competing for CPU resources in shared clusters [1]–[3]. As large-scale workflows often consume thousands of CPU hours [4], [5], CPU allocation becomes more than merely an optimization problem. Such an allocation decision can have significant cost implications on the large-scale processing of genomic data. For example, suboptimal CPU allocations propagate through the pipeline, increasing computational costs.

However, automatically determining the right CPU quota remains a significant challenge due to the unique characteristics of genomics workflows. For example, genomics workflows often exhibit substantial performance variability, depending on input characteristics. As shown in Fig. 1a, increasing input size causes execution times for some genomics applications (*e.g.*, STAR [6]) to scale linearly, whereas others (*e.g.*, GATK-br[1]) can experience up to a 10× increase in runtime. This increase in workflow execution time often results from dynamic



(a) Relative Exec. Time ↑ (b) DNA-Seq. (c) RNA-Seq.

Fig. 1. **Input-dependent performance variability.** *Genomics applications exhibit varying runtime growth with input size, up to 10× differences (1a) and shifting critical paths in DNA-Seq (1b) and RNA-Seq (1c).*

changes in the workflow's critical path, which is the sequence of operations that dominates total runtime, depending on the input. As shown in Fig. 1b and 1c, an application that takes $\leq$ 10% of runtime on small inputs may consume up to 60% on larger inputs.

Furthermore, since multiple workflows run concurrently in production environments [7], [8], contention-induced slowdowns have become increasingly difficult to detect and mitigate. Under contention, traditional CPU utilization metrics often fail to accurately capture true CPU demand. These metrics may report low utilization during I/O stalls or millisecond-scale idle bursts. I/O stalls occur when threads are blocked, waiting for disk or network operations to complete. Millisecond-scale idle bursts occur when applications rapidly alternate between computation and I/O phases at fine time scales, resulting in rapid fluctuations in CPU usage. Therefore, these patterns can make utilization metrics misleading, as they may indicate low CPU usage even when applications have pending computational work. Finally, genomics workflows can include conditional branches, where certain applications run only if the input data meets specific conditions [7], [9]. For example, low-quality genomics data triggers additional refinement steps using data quality control tools (*e.g.*, FastQC[2], Trimmomatic [10]), whereas high-quality data bypasses these stages entirely. As a result, resource demands can shift suddenly and unpredictably during execution.

Current CPU allocation approaches, *e.g.*, utilization-based CPU allocators [11]–[13], ML-based predictors [14], [15], or kernel-integrated approaches [16], [17], are often inadequate for genomics workloads. These approaches either react with a delay to dynamic CPU demands [18], [19] or require extensive historical data for predictive model training [20],

[1] https://gatk.broadinstitute.org/

[2] https://www.bioinformatics.babraham.ac.uk/projects/fastqc/

[21]. Such data *may be infeasible* to obtain given the high variability of genomic input data. Other approaches demand application/kernel-level modifications, which are incompatible with existing bioinformatics tools.

To understand these challenges quantitatively, we conduct an in-depth characterization study of widely used computational genomics workflows, including DNA sequencing (often abbreviated to DNA-Seq) and RNA sequencing (RNA-Seq) [22]. Our measurements reveal three key characteristics. Specifically, we identify *inflatable parallelism*, where thread counts surge up to 96×; measure *stall heterogeneity*, causing up to 2.1× slowdown from resource contention; and observe *millisecond-scale idle bursts*, which make utilization metrics inherently unreliable for quota decisions under contention.

These findings guide us to establish three key requirements for managing CPU quotas in genomics workflows. A solution must be (i) **responsive to thread bursts**, responding immediately to inflatable parallelism without waiting for (or confirming) utilization changes; (ii) **robust under contention**, maintaining accurate quota decisions despite heterogeneous stalls and idle bursts; and (iii) **adaptive to workflow dynamics**, adjusting to conditional branches and varying application sets without manual intervention.

We present GENOX (GENOmics eXecution), an automated CPU scaling system that addresses these requirements by exploiting thread signals as the primary indicator of compute demand. Our key insight is that thread count can serve as a primary indicator of resource needs in genomics applications, providing immediate, stall-agnostic signals that remain stable even under contention. GENOX is composed of three components: (1) a **thread-based scaler**, which allocates CPU quota proportionally to thread counts, detecting CPU demand phase changes immediately as threads are created; (2) a **fine-grained thread monitor**, which operates at 5 ms intervals to capture millisecond-scale dynamics while remaining robust against misleading utilization signals; and (3) a **container-native controller**, which seamlessly incorporates with genomics workflows/applications via standard cgroup interfaces.

We evaluate GENOX in production-like environments using widely adopted genomics workloads, including microbenchmarks with representative applications and macro-scale experiments under realistic contention scenarios. Our microbenchmarks with three representative applications (*e.g.*, BWA [23], STAR, BamSorMaDup [24]) confirm that GENOX effectively addresses all three core requirements (responsiveness, robustness, adaptiveness). GENOX reacts immediately to thread bursts in BWA's inflatable parallelism phases, maintains stable allocations despite STAR's millisecond-scale idle bursts, and efficiently adapts to BamSorMaDup's constant parallel-thread execution, collectively validating our design principles.

Moreover, macro-level evaluation shows that GENOX consistently outperforms state-of-the-art baselines [11]–[13] under realistic contention, achieving up to 50% reduction in execution time and minimizing straggler slowdowns that reach 200% in baseline approaches. At the workflow level, GENOX improves end-to-end workflow completion time by over 20%.

This work has the following research contributions.

1) **Comprehensive characterization of genomics workflow dynamics**, identifying inflatable parallelism, stall heterogeneity, and millisecond-scale idle bursts as fundamental challenges for CPU allocation (§III).
2) **Design of GENOX, a measurement-driven CPU quota allocation system** that uses threads as a stable, leading signal of compute demand, with fine-grained monitoring at 5ms intervals to detect true demand while minimizing resource-demand noise (§IV-B, §IV-C).
3) **Non-invasive, production-ready implementation** that integrates containerized workflow engines using only standard cgroup interfaces (§IV-D).
4) **End-to-end evaluations of GENOX** on production-level DNA-Seq and RNA-Seq workflows under realistic contention, confirming performance improvements over utilization-based approaches (§V).

II. BACKGROUND AND MOTIVATION

Modern genomics workflows integrate dozens of containerized bioinformatics tools into pipelines that process terabytes of biological data. Workflow engines (*e.g.*, Nextflow[3], CWLtool) coordinate task execution, while containers provide resource isolation [1], [7], [25]. We outline how such workflows operate in production environments and why CPU quota allocation is a key performance concern.

A. Genomics Workflow Background

Workflow. A genomics workflow consists of interdependent steps (*e.g.*, alignment [26], variant calling [27], annotation [28]) represented as a DAG [9]. Nodes are containerized applications, and edges are file-based dependencies. Workflows are described in languages, *e.g.*, CWL[4], WDL[5], or Nextflow DSL, which support optional inputs and conditional branches, allowing the same workflow definition to activate different steps depending on the dataset.

Job. A job is a single workflow execution with a specific dataset. Optional inputs and conditional branches enable identical workflow definitions to produce different execution paths.

Deployment. Production workflows are executed by workflow engines (*e.g.*, Nextflow, Cromwell[6], CWLtool[7]) with container runtimes (Docker, Singularity). Workflow engines schedule tasks and manage dependencies, while containers enforce CPU and memory quotas via Linux cgroups, which we leverage for dynamic and runtime CPU allocation.

Input. Genomics workflows process two main types of input: reference files (*e.g.*, the human genome) and sequencing data files such as FASTQ or BAM, typically accompanied by metadata (*e.g.*, quality scores). Input characteristics directly affect workflow execution. Data size determines computational

[3] https://www.nextflow.io/
[4] https://www.commonwl.org/
[5] https://openwdl.org/
[6] https://cromwell.readthedocs.io/
[7] https://github.com/common-workflow-language/cwltool

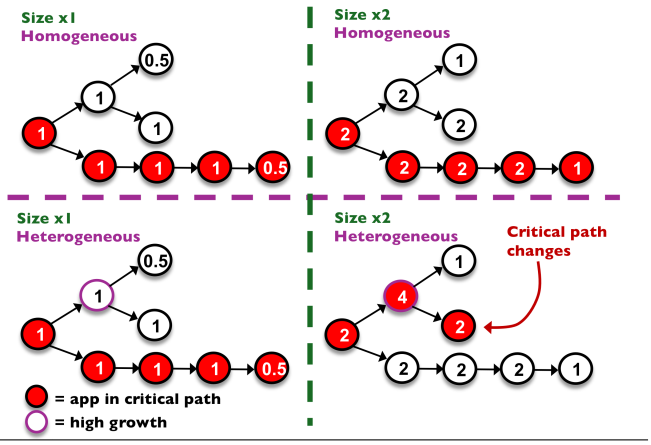


Fig. 2. **Critical path shifts with input size increase.** *'Top': Homogeneous scaling maintains the same critical path (as input size doubles). 'Bottom': Heterogeneous scaling causes the critical path to shift between applications (from blue to purple), complicating CPU quota allocation.*

requirements, format affects I/O patterns, and optional inputs enable conditional branches that activate different subsets of applications [7], [9]. We further discuss how this variability complicates resource prediction in §III-A.

Critical path. A workflow's completion time depends on its critical path, the longest chain of dependent steps in the DAG [29]. In production environments with input-dependent dynamic branching, the same workflow definition may activate different steps for different inputs, resulting in shifting critical paths that complicate CPU quota allocation.

Alignment workflow. Alignment matches DNA or RNA sequencing reads to a reference genome [30] and is among the most resource-intensive stages in genomics, often dominating workflow runtime due to preprocessing, quality control, and the batch execution patterns common in production [4].

B. Container-based Deployment Practices

Different production systems that process genomic workflows work in different ways. For example, Tong et al. [4] describes the GDC GPAS system that was developed during the 2018-2020 period. During this time, CWL was one of the dominant workflow languages in the genomics research community and CWLtool was the most reliable open source workflow engine for CWL pipelines. However, CWLtool did not provide good support for parallelism. For this reason, and other reasons, the GPAS system ran one CWL workflow per node. The tasks in the workflow were containerized. Some of the tools within a single CWL workflow would themselves use parallelism so that at times multiple instances were running on the node in parallel.

In contrast, today Nextflow is one of the main workflow engines used by the genomics community, which provides sophisticated support for parallelism with a single pipeline. Typically, tasks in a Nextflow workflow are also containerized. This paper is targeted at production systems that use engines like Nextflow where there is a high degree of parallelism on a single node.

C. CPU Quota Allocation in Production

Genomics workflows execute on shared clusters where multiple workloads compete for CPU resources. Current allocation schemes struggle with both over-allocation (wasting resources) and under-allocation (slowing down executions due to insufficient resources) problems, compounded by genomics workflows' runtime variability. Existing systems employ either static allocation, where resources are fixed at submission time (*e.g.*, SLURM [31], PBS [32], [33]), or dynamic allocation, where CPU quotas are adjusted based on utilization (*e.g.*, Kubernetes[8], YARN [34]). However, both fail to handle genomics-specific challenges. For example, static allocation cannot adapt to runtime variations, and utilization-based dynamic approaches suffer from inherent lag during demand bursts.

Given these limitations, an important question arises: *within a single compute node (with the fixed total number of CPU cores), what CPU allocation control knob actually remains?* To answer this, we need to look at how genomics workloads are actually being executed in production. Cluster schedulers pack dozens of containerized genomics tasks onto each compute node, and once placed, these containers remain pinned to that node for hours or days [4]. Migration is impractical as genomics tasks carry heavy state (multi-gigabyte reference indexes, intermediate BAM files, and partially processed sequencing reads). Within these constraints, dynamic CPU quota adjustment within the node is the remaining control knob. In §III, we conduct a measurement study with production genomics workflows to guide how this knob should be controlled.

III. Challenges and Measurement

The unique runtime characteristics of genomics workflows present significant challenges for effective CPU quota allocation. We conducted measurements on two representative genomics workflows: DNA-Seq and RNA-Seq. We use Nextflow workflows that we develop ourselves based on GDC's open-source CWL workflows [22] and with the help of a semi-automatic CWL-to-Nextflow workflow translator [1]. For each workflow, we use only the 10 longest-running applications, including widely used genomics applications (*e.g.*, BWA, STAR, GATK, Trimmomatic) that dominate the runtime of production workloads, while maintaining the original DAG order of the workflows. The application versions are as follows: BWA v0.7.15, Samtools v1.20, Picard v2.26.10, STAR2 v2.7.10b, GATK v4.2.4.1, FastQC v0.11.7, Trimmomatic v0.38, and BamSorMaDup v2.0.185. Our measurements show that our Nextflow workflows are good representatives of the original CWL workflows – they capture 95% of the original workflows' execution time with an almost identical resource usage pattern.

Meanwhile, for input, we use the genome assembly GRCh38 as reference files. Sequencing data files are obtained from the Sequence Read Archive (SRA)[9], such as

[8] https://kubernetes.io/
[9] https://www.ncbi.nlm.nih.gov/sra

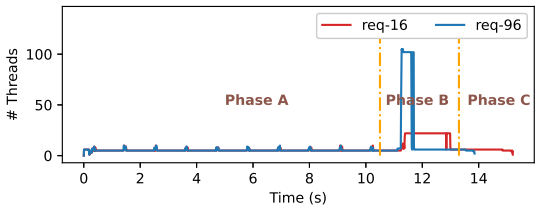


Fig. 3. **Inflatable parallelism ('Phase B') in BWA execution.** *Thread count dynamics for two configurations (16 and 96 threads). Phase A: I/O-driven spikes, Phase B: thread surges proportional to user config., Phase C: a stable thread level until completion. Phase B reveals configuration's impact on resource consumption and execution time.*

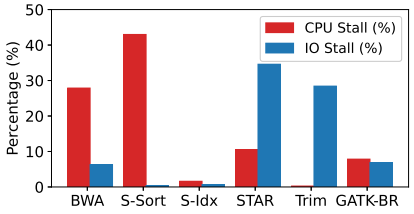


Fig. 4. **Stall heterogeneity across genomics applications.** *Distribution of CPU and I/O stall percentages showing diverse resource bottlenecks. Some applications are predominantly CPU-bound while others are I/O-bound, complicating utilization-based allocation under contention.*

SRR17250075 for DNA-Seq and SRR6490047 for RNA-Seq. All sequencing data used are publicly accessible. Finally, we control the absolute sizes of sequencing data up to 2 GB by splitting the original FASTQ files.

We executed the workflows on the Chameleon Cloud, using servers with 96 CPU cores and 192 GB memory. Nextflow orchestrated execution with each application running in Docker containers. We monitored runtime metrics through kernel interfaces, collecting per-container CPU quotas, thread counts, and pressure stall information (PSI) [35]. We varied dataset sizes up to 12× to capture input-dependent effects.

Based on these measurements, we reveal three key challenges: (i) **input-dependent performance variability** (§III-A), (ii) **contention-induced slowdown** (§III-B), and (iii) **dynamic workflow branching** (§III-C). For each challenge, we conduct measurements to quantify its impact, reveal unique execution patterns, *e.g.*, *inflatable parallelism* and *stall heterogeneity*, and derive design insights that motivate our solution.

A. Input-dependent Performance Variability

Resource allocation for workflows should prioritize applications on the critical path to minimize their execution time. However, accurately predicting the completion time of genomics workflows remains challenging due to their complex execution characteristics. In particular, as input size increases, some workflows exhibit linear scaling, while others show up to a 10× increase in runtime beyond what linear scaling would predict. This heterogeneous, non-linear scaling shifts the critical path across applications, making it difficult to identify which applications will dominate runtime and complicating decisions for CPU quota allocation.

Fig. 2 explains this phenomenon conceptually. The top cases depict *homogeneous growth*, where all applications scale uniformly and the critical path remains unchanged as input size doubles. By contrast, the bottom cases depict *heterogeneous growth*, where differential scaling causes the critical path to shift to a different application. This heterogeneous behavior is what we have observed in production genomics workflows.

To quantify this challenge, we analyzed the 5 longest-running applications in both DNA-Seq and RNA-Seq workflows, measuring execution time growth under identical input size increases. Our results (Fig. 1 in §I) reveal that application run times increased by 22% to 232% relative to their baseline, with some experiencing up to 10× greater increase than others (Fig. 1a). This uneven scaling shifts the critical path (the application dominating runtime) as input size grows (Fig. 1b, 1c). In production environments where inputs reach hundreds of GB, these shifts become pronounced and further complicate CPU quota allocation.

Finding #1: Inflatable parallelism. Our measurements reveal that parallel genomics applications (*e.g.*, BWA) exhibit distinct execution phases corresponding to different computational stages. We consistently observe a phase with sharp thread count surges, where threads increase dramatically based on user configuration. We introduce the term *inflatable parallelism* to describe this behavior, where thread counts can suddenly expand based on user configuration, creating unpredictable resource demands. Fig. 3 illustrates this behavior for BWA, which proceeds through three phases: *Phase A*: recurrent small spikes, coinciding with I/O activity; *Phase B*: inflatable parallelism stage with thread surge proportional to the user configuration; *Phase C*: a drop followed by a stable number of threads until completion.

Only *Phase B* is sensitive to the user-specified configuration, causing variations in thread counts and overall execution time. This inflatable parallelism poses a significant challenge. *While proportional scaling may seem predictable, its impact on workflow runtime is substantial.* The inflated phase often dominates execution time and can shift the critical path as inputs grow. Existing utilization-based policies fail to timely detect these sudden inflations because utilization lags behind thread creation.

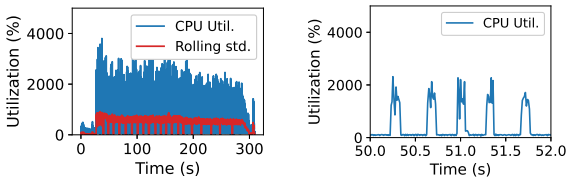
B. Contention-induced Slowdown

Genomics workflows are often executed in batches [4], leading to the colocation of multiple applications on the same machine. Such colocated executions inevitably create contention, causing slowdowns as applications interfere with one another's resource usage. While resource allocators should mitigate this contention, genomics applications exhibit two characteristics that make traditional utilization-based detection ineffective: (i) *stall heterogeneity* and (ii) *millisecond-scale repetitive idle bursts*.

Stall heterogeneity. Genomics applications differ widely in their dominant resource usage. Some applications are CPU-bound, others I/O-bound, and many alternate between the two. Fig. 4 shows the distribution of CPU and I/O stall time

Table 1. **Contention-induced slowdowns across application pairs.** *Executed on Linux with no quota set. Entries show normalized execution time; larger values indicate higher slowdown. Trimm.: Trimmomatic, P-DNA: Picard-DNA, P-RNA: Picard-RNA*

	Trimm.	STAR	P-RNA	BWA	GATK-br	P-DNA
Trimm.	1.4×	–	–	–	–	–
STAR	1.1×	2×	–	–	–	–
P-RNA	1.03×	1.01×	1.08×	–	–	–
BWA	1.07×	1.1×	1.05×	2.1×	–	–
GATK-br	1.08×	1.05×	1.05×	1.13×	2.02×	–
P-DNA	1.15×	1.04×	1.02×	1.11×	1×	1×



(a) CPU Utilization (b) Zoomed 2-sec. Window

Fig. 5. **Millisecond-scale idle burst dynamics in GATK.** *(CPU utilization in Linux %CPU; 100% = one core). (a): CPU utilization shows large volatility over time. (b): fine-grained oscillations from alternating compute and I/O phases.*

as reported by Linux PSI. Depending on the application, between 10% and 50% of execution time can be lost to stalls on different resources. This heterogeneity makes utilization an unreliable signal. For example, a high utilization reading may indicate genuine compute demand that would benefit from more CPUs, or it may simply be inflated by I/O-induced stalls and scheduling delays, in which case allocating additional CPUs would not improve performance. When such applications are colocated, interference leads to non-uniform slowdowns. Table 1 shows that co-execution slows down genomics workflows by up to 2.1×.

Millisecond-scale repetitive idle bursts. Genomics applications frequently interleave compute-intensive phases with short I/O operations, producing millisecond-scale repetitive idle bursts. Fig. 5a shows GATK's CPU utilization fluctuating sharply with 500% – 800% standard deviation. These bursts are not genuine demand shifts but the result of applications rapidly alternating between CPU-intensive computation and I/O-bound waiting (Fig. 5b), which appears in utilization measurements as oscillation despite stable underlying demand. Under contention, such bursts further destabilize utilization as a scaling signal, creating the illusion of rapid demand changes.

Finding #2: Utilization misleads under contention. Taken together, these two characteristics (stall heterogeneity and millisecond-scale repetitive idle bursts) suggest that utilization is an unreliable indicator when applications are colocated. It cannot distinguish compute demand from I/O stalls, nor differentiate genuine idle periods from brief I/O waits. This finding necessitates a stall-agnostic, phase-aware signal for robust quota allocation.

C. Dynamic Workflow Branching

Finally, resource allocation for genomics workflows faces an additional challenge: dynamic workflow branching. Workflows often contain conditional branches that activate or skip certain applications based on specific input characteristics. These branches arise from conditional logic triggered by input type, input characteristics, input existence, or input metadata. Developers incorporate such logic to selectively perform pre-/post-processing or to tune algorithms based on input characteristics.

Our analysis of widely used harmonization workflows [22] shows that such branching is common, and its runtime impact varies with workflow complexity. In DNA-Seq workflows, we observed that a conditional Picard step adds about 5% to execution time. While modest in isolation, the effect compounds in production workflows with multiple conditional steps, making accurate resource prediction increasingly difficult.

Finding #3: Dynamic thread demand from conditional execution. The set of active applications can change at runtime based on input characteristics, causing workflows to exhibit shifting resource demands and thread dynamics that cannot be predicted a priori. This unpredictability undermines static allocation strategies and necessitates runtime adaptivity.

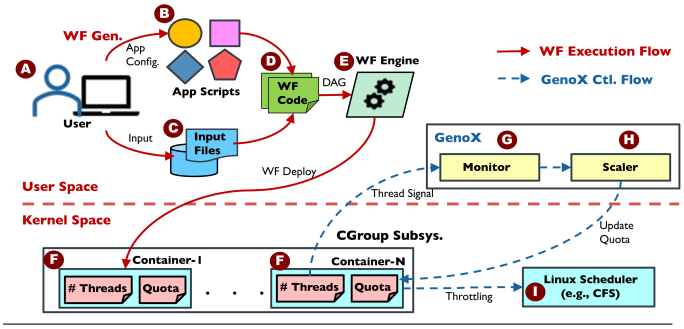


Fig. 6. **GENOX architecture.** GENOX *operates alongside the workflow engine as a sidecar: its monitor collects thread counts, the scaler computes CPU quotas, and quotas are enforced via Linux cgroups and the CFS scheduler while applications run inside containers.*

D. Measurement Summary and Design Insights

These measurements reveal that utilization-based policies are fundamentally limited for genomics workflows. To address this, an effective CPU allocator must satisfy three requirements: **responsiveness** to thread bursts (§III-A), **robustness** under stall heterogeneity and idle bursts (§III-B), and **adaptiveness** to dynamic workflow branching (§III-C). The following section presents GENOX, our design that meets these requirements through thread-based signals.

IV. GENOX DESIGN

We design GENOX to address the challenges in §III. As shown in Fig. 6, GENOX operates as a sidecar alongside the workflow engine, monitoring genomics applications and dynamically adjusting CPU quotas at runtime. It includes three core components: a thread-based scaler (§IV-B), a fine-grained monitor (§IV-C), and a container-native controller (§IV-D).

A. GENOX Architecture Overview

Fig. 6 illustrates how GENOX integrates into end-to-end genomics workflow execution. While the workflow engine

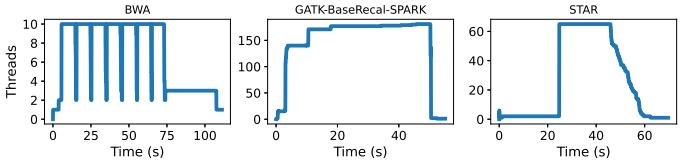


Fig. 7. **Thread-level dynamics in genomics applications.** *Thread counts shift sharply across phases, exposing inflatable parallelism that utilization signals fail to capture.*

launches containerized applications from user-provided scripts and inputs, GENOX runs alongside as a sidecar: its monitor collects per-container thread signals at 5 ms intervals, the scaler computes the proportional CPU quotas (approximately one core per thread), and the kernel's CFS scheduler enforces them via standard cgroup interfaces without modifications to the workflow engine or applications. The next sections describe each component.

B. Thread-based Scaler for Responsiveness

Choosing the right scaling signal is crucial to GENOX design, as the signal determines when and how much CPU allocation should change to match compute demand. The signal must accurately reflect true demand so GENOX can allocate resources proportionally without performance loss.

Thread count as a scaling signal. GENOX uses thread count as its primary scaling signal, providing immediate response to phase changes. Unlike utilization-based policies that lag behind actual demand, thread count changes immediately when an application enters a new phase. This enables GENOX to detect *inflatable parallelism* (§III-A), where applications spawn additional threads in proportion to user-specified settings. By tracking thread counts directly, GENOX responds to bursts promptly at fine granularity. Fig. 7 shows how thread counts clearly delineate execution phases in representative genomics applications (*e.g.*, BWA, STAR, and GATK).

While there are other metrics which reflect CPU contention and thus have potential as scaling signals, such as nr_running or PSI, these metrics have inherent limitations when used in our setting. For example, nr_running does not directly map to a particular cgroup, which is necessary to determine allocation for individual running container. PSI's "avg10/avg60/avg300" provide moving averages and do not accurately reflect true demand in real time. PSI also has a "total" metric which is updated in real time, but although it might be a good signal for increasing CPU allocation, it does not provide any information on how to reduce allocation once demand is met. In contrast, thread count can be directly tracked per cgroup and is responsive towards applications' rapidly changing demand.

Mechanism. To implement this, GENOX obtains two kernel-level metrics for each container: the number of threads and the assigned CPU quota, both accessible through standard cgroup interfaces (*e.g.*, '`/sys/fs/cgroup`') with negligible overhead. The scaler then applies an empirically validated proportional heuristic that allocates approximately one CPU core per existing thread. This approach balances responsiveness and control overhead while aligning with the Linux scheduler's granularity. For example, if ten threads exist, the application receives a quota for ten cores; if only two threads remain, the quota is reduced accordingly. Note that idle or I/O-blocked threads within this allocation impose no actual cost. The Linux CFS scheduler immediately yields their cores to runnable tasks elsewhere, so the quota functions as a cap on potential demand rather than a reservation. There is also no feasibility issue when aggregate quotas exceed node capacity (*e.g.*, due to large number of running containers) or fall well below it. In the former case, GENOX leverages the built-in capability of Linux CFS scheduler to resolve excess quotas and allocate CPU time across containers. In the latter case, since a running container will have at least one thread, GENOX will always allocate at least one core. The leftover cores will stay available.

The monitor and scaler operate as a producer-consumer loop forming GENOX's core feedback mechanism. The monitor (producer) samples per-container thread counts at short intervals, while the scaler (consumer) recomputes quotas accordingly. This design ensures timely, low-overhead adjustments, as thread counts are accessed efficiently via cgroup interfaces. The same feedback pattern extends to contention handling in the fine-grained monitor (§IV-C).

Impact. By leveraging thread-level signals, GENOX captures phase shifts immediately and handles inflatable parallelism without the lag of utilization-based policies.

C. Fine-grained Thread Monitor for Robustness

When multiple genomics jobs execute concurrently, contention for shared resources leads to performance variability. Some phases are compute-bound while others are dominated by I/O stalls, creating heterogeneous stall patterns and millisecond-scale idle bursts (§III-B). For example, short idle bursts can make utilization appear low even though applications continue to demand CPU time. Utilization-based policies that rely on this unstable signal risk under-allocating quota during critical phases. To ensure robustness, GENOX monitors thread counts as a stable signal that is unaffected by whether stalls originate from computation or I/O.

Mechanism. This fine-grained monitor reuses the core feedback loop from §IV-B, but is more specialized for high-frequency sampling under contention. The monitor records per-container thread counts at a 5 ms interval, which is short enough to capture rapid idle bursts while remaining lightweight. In particular, this 5 ms interval is a trade-off. Each sampling period requires the monitor to perform two reads and one write operation to a container's cgroup pseudo-filesystem. Shorter intervals (*e.g.*, 1 ms) may introduce non-trivial monitoring cost, while longer ones (*e.g.*, 100 ms) miss the millisecond-scale bursts (§III-B). For example, assuming the read/write operations take 10μs per container, with 20 running containers and a 1 ms interval, the monitoring will cost $\frac{0.01\times 20}{1} \times 100\% = 20\%$ single-core utilization.

Because threads remain visible even when blocked on I/O, thread counts provide a stall-agnostic and timely reflection of

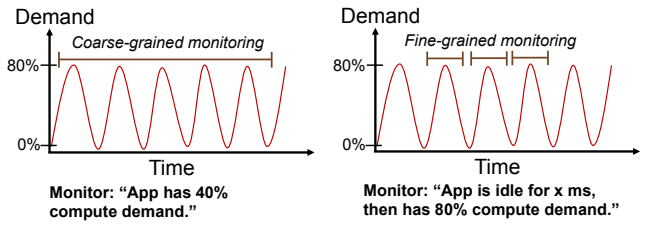


Fig. 8. **Coarse-grained vs. fine-grained monitoring.** *Applications differ widely between CPU- and I/O-bound profiles, making utilization an unreliable signal under contention.*

demand. These measurements are continuously streamed to the scaler for quota adjustment. Fig. 8 illustrates this behavior: while coarse-grained monitoring aggregates idle bursts with compute phases making utilization appear moderate, fine-grained monitoring at 5 ms intervals captures the true demand pattern. Thread counts remain stable even as utilization oscillates, enabling GENOX to base decisions on reliable signals.

Impact. By grounding quota decisions in stable thread-level signals, GENOX avoids misinterpreting idle bursts and sustains fair allocation under contention.

D. Container-native Controller for Adaptiveness

Genomics workflows frequently undergo dynamic branching (§III-C), where new containers may be created or terminated on-the-fly. A static quota plan cannot anticipate such changes, leading to wasted resources or starvation. To remain adaptive, GENOX must handle workflow dynamics without manual intervention or workflow engine modifications.

Mechanism. GENOX achieves this through a container-native controller that integrates with Linux cgroups. When a workflow engine launches new tasks, they appear as containers in the cgroup hierarchy; the controller detects them in real time and applies GENOX's quota policies without workflow-level awareness. When containers terminate, their quotas are immediately returned to the pool.

The controller operates as a lightweight daemon that monitors the cgroup subsystem for container events. Upon detecting a new container, it attaches quota enforcement hooks so that scaling decisions from the monitor and scaler (§IV-B, §IV-C) are applied transparently. Because it leverages existing OS primitives, the controller incurs negligible overhead and requires no modifications to workflow engines or applications.

Impact. By coupling quota enforcement, GENOX adapts to new and completed tasks throughout branching workflows, without modifications to applications or workflow engines.

V. EVALUATION RESULTS OF GENOX

We evaluate GENOX in production-like settings using production-scale datasets and complete DNA-Seq and RNA-Seq pipelines. Our evaluation assesses three criteria: (i) **responsiveness** to bursty, phase-shifting demand, (ii) **robustness** under contention and I/O stalls, and (iii) **adaptiveness** to workflow dynamics.

Evaluations cover micro-benchmarks (case studies, §V-B) and macro-scale (§V-C, §V-D) results. The case studies validate GENOX's mechanisms on representative applications, and macro-scale experiments quantify application-level and workflow-level improvements under realistic contention.

Connecting characterization to evaluation. We first map each experiment to one of the three challenges in §III, since each evaluation deliberately emphasizes specific findings rather than treating the challenges as an abstract motivation. §V-B isolates individual challenges on representative applications, *i.e.*, BWA exercises the inflatable parallelism (§III-A), STAR exercises both the millisecond-scale idle bursts and the stall heterogeneity (§III-B), and BamSorMaDup serves as a steady-state control with neither of these patterns active. §V-C then exposes GENOX to all three challenges simultaneously under realistic contention, where the stall heterogeneity (§III-B) drives the straggler dynamics that distinguish thread-based scaling from utilization-based baselines. Finally, §V-D executes genomic pipelines whose conditional branches activate different application sets at runtime, which is the dynamic workflow branching (§III-C). Because GENOX uses runtime thread signals to make scaling decisions, it adapts to whichever branch is taken without per-dataset tuning.

A. Evaluation Methodology

HW setup. All experiments are conducted on the Chameleon NSF Cloud testbed [36], using a server equipped with 96 CPU cores, 192 GB of memory, and 512 GB of SSD storage. Workflows are containerized and orchestrated via Nextflow. We use the DNA-Seq and RNA-Seq workflows and inputs described in §III.

Baselines. We evaluate GENOX against three baselines, which are widely studied in dynamic CPU allocation.

1) *Autopilot* [11] monitors recent CPU utilization based on its moving-window averaging strategy and adjusts quotas based on smoothed historical demand. This smoothing avoids overreacting to momentary noise, but it also delays responses to sudden bursts or phase shifts. 2) *AutoThrottle* [12] employs reactive scaling to adjust CPU quotas aggressively once utilization crosses predefined thresholds. AutoThrottle responds quickly to bursts but is prone to overshoot and instability when demand fluctuates. 3) *SHOWAR* [13] uses statistical thresholding (*e.g.*, three-sigma rules) to identify deviations from average utilization. SHOWAR aims to be robust against noise but often underestimates demand in bursty workloads, leading to slowdowns due to stragglers.

All GENOX's artefacts are available at https://github.com/martinluttap/GenoX-Artifacts.

Evaluation metrics. We use metrics at both application and workflow levels. For application-level studies, we measure per-application execution time, normalized speedup, and straggler slowdown. To capture straggler effects accurately, we evaluate the completion time of the last-finishing workflow, reflecting the cumulative impact of contention across constituent applications. For workflow-level studies, we measure end-to-end

Table 2. CPU usage patterns of three representative genomics applications. *Each application is selected to demonstrate a particular design choice of* GENOX*, i.e., responsiveness (BWA), responsiveness and robustness (STAR), and adaptivity (BamSorMaDup).*

Application	Exec. Pattern (Thread/CPU)	Eval. Focus
BWA	Bursty inflatable parallelism with sudden thread inflation	Responsive.
STAR	Multi-phase execution with rising /falling threads and idle gaps	Responsive. + Robust.
BamSor-MaDup	Constant parallel execution with fixed 16-thread demand	Adaptive.

runtime and speedup of complete DNA-Seq and RNA-Seq pipelines, showing how per-stage allocation impacts overall throughput in production workflows.

B. Micro-benchmarks (Case Studies)

To validate GENOX's mechanisms under controlled execution patterns, we conduct microbenchmarks on three representative genomics applications: BWA, STAR, and BamSorMaDup. Table 2 summarizes how each application corresponds to the challenges (in §III) and evaluates a specific GENOX's capability: responsiveness, robustness, and adaptiveness.

BWA (Responsiveness). We use BWA, a DNA aligner [23], to stress GENOX's ability to handle *bursty inflatable parallelism* (§III-A). The key question is whether GENOX can adjust quotas fast enough to prevent throttling during bursts.

Fig. 9a shows CPU utilization under different policies. Utilization-based approaches (Autopilot, SHOWAR, AutoThrottle) lag behind bursts, reacting only after resource consumption changes. GENOX prevents throttling by tracking threads directly. Fig. 9b reports CPU quota allocation over time. GENOX increases CPU quota promptly with thread inflation and scales down once bursts decline, while baselines react only after demand has shifted, causing temporary under-allocation. As a result, GENOX minimizes throttling penalties and achieves the fastest completion time among all policies (Fig. 9c).

STAR (Responsiveness + Robustness). We use STAR, an RNA-seq aligner [6] to test GENOX under *multi-phase execution with idle bursts and heterogeneous stalls* (§III-B). The key challenge is whether GENOX can remain responsive to bursts while being robust against misleading I/O stall signals.

Fig. 10a shows CPU utilization across policies. Baseline approaches overestimate demand during idle bursts and underestimate it when demand rises quickly. However, GENOX, relying on thread counts rather than utilization, avoids these misinterpretations and maintains a closer alignment with actual demand. Fig. 10b reports quota allocation. GENOX scales quotas up immediately as thread counts rise and reduces them once phases complete. In contrast, baselines lag behind demand by adjusting CPU quota only after consumption has shifted. As a result, GENOX achieves the fastest completion time (Fig. 10c). By preventing over-throttling during bursts and avoiding over-provisioning during idle gaps, GENOX reduces wasted resources and improves efficiency relative to all baselines.

BamSorMaDup (Adaptiveness). We use BamSorMaDup, a widely-used duplicate-marking tool [24], to evaluate GENOX's *adaptiveness under steady parallel workloads*, where the thread count remains constant at 16 throughout execution (§III-C). The challenge is whether GENOX can avoid wasting resources in the absence of bursts or dynamic branching.

Fig. 11a shows CPU utilization. Baseline policies occasionally misinterpret I/O stalls, leading to oscillations in quota allocation. GENOX, by contrast, detects the constant 16-thread demand and keeps allocations minimal and stable. Fig. 11b reports CPU quota allocation. GENOX consistently assigns one core per thread (16 cores total), matching the constant-thread profile. However, the baselines make unstable quota adjustments due to noisy utilization signals, leading to inefficient resource allocation. As shown in Fig. 11c, all policies finish within a similar time frame since BamSorMaDup is not CPU-intensive. However, GENOX achieves this without wasting CPU cycles, demonstrating its efficiency and adaptiveness.

C. Application-level Performance

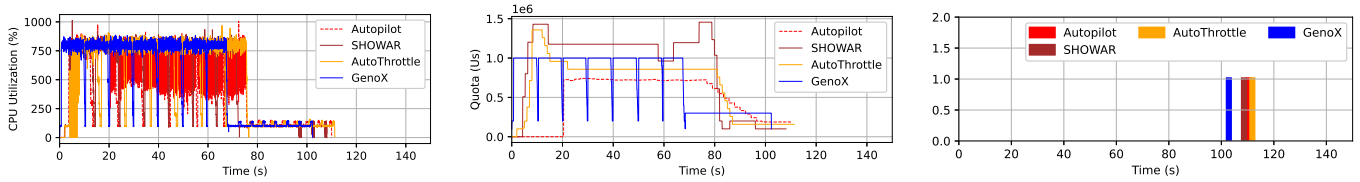
We evaluate GENOX at the application level under resource contention. We execute two concurrent workflows, each running several genomics applications that compete for shared resources. This reflects production environments where co-running tasks and straggler effects are prevalent. Performance is measured as the completion time of the last-finishing workflow, capturing contention across all constituent applications.

Fig. 12a shows the execution times of DNA-Seq applications under contention. GENOX consistently achieves the shortest completion times and outperforms all baseline methods by effectively adapting to varying CPU usage patterns across applications. In contrast, baseline approaches show inconsistent performance across applications. For example, while SHOWAR achieves the second-best performance for Trimmomatic, it performs worst for BWA. Similarly, Autopilot ranks second-best for Samtools-Index but exhibits the poorest performance for Samtools-Sort. Across all evaluated applications, GENOX achieves the fastest completion times, reducing execution time by up to 49% relative to baseline methods, with an average improvement of 15.7%.

Fig. 12b presents straggler slowdowns, defined as the extra wait time of slowest-workflow applications relative to the fastest (*e.g.*, if the fastest finishes at $t = 8$ and the slowest at $t = 10$, the slowdown is $(10 - 8)/8 = 25\%$). GENOX effectively avoids such slowdowns, while utilization-based baselines suffer significant delays. SHOWAR exhibits nearly 200% slowdown on BWA, where the faster workflow waits almost its entire runtime, and also slows GATK-BR and STAR. Autopilot is better but still incurs 43% slowdown on BWA and 23% on Samtools-Sort. In contrast, GENOX prevents straggler accumulation through prompt quota adjustments.

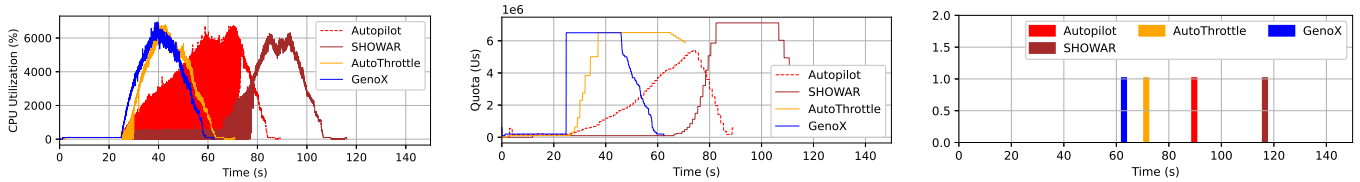
D. Workflow-level Performance

We execute complete DNA-Seq and RNA-Seq pipelines with production-scale datasets to measure GENOX's workflow-level performance. This evaluation captures the cumulative



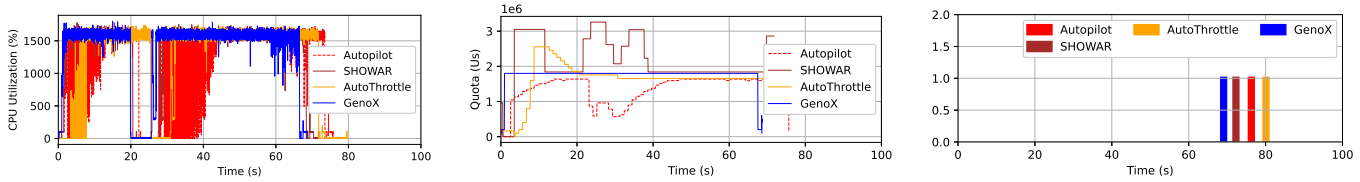
(a) CPU utilization (BWA) (b) CPU quota allocation (BWA) (c) BWA finish times across policies

Fig. 9. **BWA case study results.** *'(a) CPU utilization (100% = one core):' Baselines lag behind bursts while* GENOX *immediately detects thread inflation. '(b) Quota allocation over time (Y-axis in 10^6 µs; each unit equals 10 cores):'* GENOX *scales up/down promptly with thread count, whereas baselines react after demand shifts. '(c) Completion times:'* GENOX *achieves the shortest runtime by eliminating/minimizing throttling penalties, while baselines finish later due to delayed reactions.*



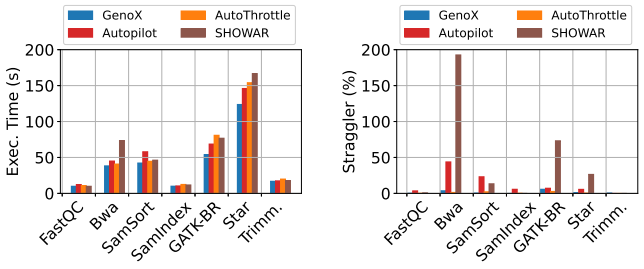
(a) CPU utilization (STAR) (b) CPU quota allocation (STAR) (c) STAR finish times across policies

Fig. 10. **STAR case study results.** *(a) 'CPU utilization (100% = one core):' Baselines misinterpret idle bursts as reduced demand while* GENOX *tracks thread counts to maintain accurate allocation. (b) 'Quota allocation over time (Y-axis in 10^6 µs; each unit equals 10 cores):'* GENOX *responds immediately to thread changes, whereas baselines exhibit delayed reactions. (c) 'Completion times:'* GENOX *achieves fastest runtime through prompt response and robustness to idle bursts.*



(a) CPU utilization (BamSorMaDup) (b) CPU quota allocation (BamSorMaDup) (c) BamSorMaDup finish times across policies

Fig. 11. **BamSorMaDup case study results.** *'(a) CPU utilization (100% = one core):' Baselines fluctuate due to I/O stalls while* GENOX *maintains stability matching the constant 16-thread profile. '(b) Quota allocation over time (Y-axis in 10^6 µs; each unit equals 10 cores):'* GENOX *consistently assigns one core per thread, whereas baselines make unstable adjustments. '(c) Completion times:' All policies finish similarly, but* GENOX *avoids resource waste, demonstrating efficient adaptation to steady workloads.*



(a) Execution Times (b) Straggler Slowdown

Fig. 12. **Execution times (12a) and straggler slowdown (12b) of DNA-Seq under contention.** *(a)* GENOX *consistently reduces completion times across applications while baselines are inconsistent. (b)* GENOX *prevents straggler delays, whereas baselines suffer severe slowdowns (e.g., 43% – 200%)*

effects of CPU quota allocation decisions across multiple workflow stages. In such pipelines, delays in a single application can propagate through dependent tasks, leading to significant increases in overall workflow runtime.

Table 3 reports total workflow execution times. GENOX consistently achieves the shortest runtimes by reducing completion time by 12% – 18% for DNA-Seq and 19% – 25% for RNA-Seq compared to baselines. At the workflow level, GENOX was able to maintain the improvements observed at the application level, with average speedups comparable to per-application benefits.

Table 3. **End-to-end execution times (sec.) for DNA-Seq and RNA-Seq.** GENOX *consistently achieves the shortest runtimes, speeding up DNA-Seq by 12 – 18% and RNA-Seq by 19 – 25% over baselines.*

	Autopilot	AutoThrottle	SHOWAR	GENOX
DNA-Seq	1085	1115	1059	944
RNA-Seq	339	352	335	282

Fig. 13 and 14 report the start and finish times for individual applications within both workflows. Baselines often prolong long-running stages, delaying dependent tasks and inflating overall runtime. For example, STAR in RNA-Seq extends by 19% under SHOWAR, while P-OxO (PicardCollectOxoG-

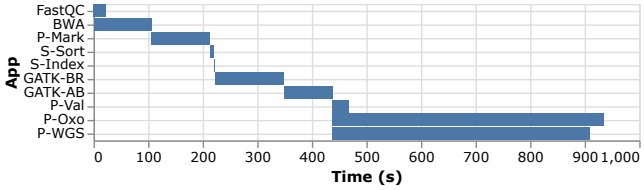


Fig. 13. **Per-application start and finish times in DNA-Seq.** *Baselines prolong key stages (e.g., P-OxO by up to 26%), delaying downstream tasks. However,* GENOX *keeps stage completion times shorter and more balanced.*

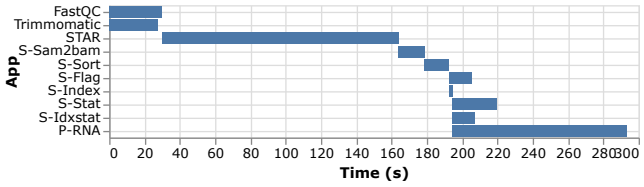


Fig. 14. **Per-application start and finish times in RNA-Seq.** *Baselines extend long-running stages such as STAR (by 19% under SHOWAR), cascading delays to dependent tasks;* GENOX *sustains faster workflow-level progress.*

Metrics)[10] in DNA-Seq is prolonged by 26%. These stage-level delays compound throughout the workflow, explaining the performance gaps in Table 3.

VI. Discussion

We discuss the scenarios in which GENOX is most effective, as well as its current limitations.

Effective Scenarios for GENOX. Our evaluation shows that GENOX achieves the highest effectiveness in dynamic workflows with significant resource contention, which are frequently encountered in genomics pipeline workloads. When applications exhibit bursty parallelism or undergo phase shifts, GENOX leverages thread-level signals to respond immediately, thereby eliminating the lag that is often observed in utilization-based policies. Its fine-grained monitor prevents idle bursts from being incorrectly interpreted as reduced resource demand, which ensures stable and accurate CPU quota allocation throughout workflow execution.

Limitations of GENOX. While GENOX is effective in many settings, its benefits *may* diminish under certain conditions.

First, GENOX's reliance on thread count as its primary scaling signal offers limited advantages for single-threaded applications. When execution is inherently serial, GENOX performs similarly to Linux defaults, as it allocates one core throughout the run. To address this limitation, we plan to incorporate additional scaling signals, such as the scheduler statistics from '`/sys/fs/`' and '`/proc/`' interfaces, which can improve demand estimation.

Second, GENOX targets CPU-based genomics pipelines (*e.g.*, BWA, GATK, STAR) and does not currently support GPU-accelerated tools. Although GPU-based implementations are gaining growing interest (*e.g.*, NVIDIA Parabricks[11]) in the genomics community, integrating GENOX's thread-based scaling with GPU runtimes raises distinct challenges, such as tracking GPU kernel concurrency rather than CPU thread counts, which we leave to future work.

[10] https://broadinstitute.github.io/picard/

[11] https://docs.nvidia.com/clara/parabricks/latest/index.html

VII. Related Work

CPU quota allocation for parallel applications has been extensively studied in HPC and cloud domains. We categorize prior work based on their allocation strategies and compare them with GENOX.

Utilization-based Reactive Approaches. Autopilot, AutoThrottle, and SHOWAR share our cgroup-based, non-invasive design but rely on utilization signals. As §V shows, they react with delay to inflatable parallelism (§III-A) and ms-scale idle bursts (§III-B), failing to handle the runtime patterns characteristic of genomics workloads.

Predictive Approaches. Nodens [14] and SmartHarvest [15] use machine learning to estimate future demand. However, they rely on substantial training data and historical patterns, which become unreliable for genomics workflows characterized by input-dependent variability (§III-A) and dynamic branching (§III-C).

Other System-level Approaches. A separate line of work modifies application or system software for finer resource control. Arachne [37] and Shenango [38] target microsecond-scale latency for microservices, requiring custom runtimes or code changes. HarvestVM [16] and PerfIso [17] operate at the hypervisor or kernel layer with coarser (tens of milliseconds) granularity. Both lines require privileged access or extensive modifications, which are impractical for container-based genomics workflows that rely on numerous third-party bioinformatics tools.

VIII. Conclusion

GENOX is a dynamic CPU quota allocator that leverages thread counts as a stall-agnostic signal of compute demand in genomics workflows. By monitoring per-container threads at 5 ms intervals and enforcing quotas through standard Linux cgroups, GENOX reduces application execution time by nearly 50%, eliminates straggler slowdowns of up to 200%, and improves end-to-end workflow runtime by over 20% compared to state-of-the-art baselines.

Acknowledgments

We thank the anonymous reviewers for their feedback and guidance. This work was supported in part by the National Science Foundation under grants 2416213 and 2416214. Results presented in this paper were obtained using the Chameleon testbed supported by the National Science Foundation.

References

[1] Martin L. Putra, In Kee Kim, Haryadi S. Gunawi, and Robert L. Grossman. CNT: Semi-Automatic Translation from CWL to Nextflow for Genomic Workflows. In *IEEE International Conference on Bioinformatics and Bioengineering (BIBE)*, 2023.

[2] Fabian Lehmann, Jonathan Bader, Friedrich Tschirpke, Lauritz Thamsen, and Ulf Leser. How Workflow Engines Should Talk to Resource Managers: A Proposal for a Common Workflow Scheduling Interface. In *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2023.

[3] Md. Vasimuddin, Sanchit Misra, Heng Li, and Srinivas Aluru. Efficient Architecture-Aware Acceleration of BWA-MEM for Multicore Systems. In *IEEE International Parallel & Distributed Processing Symposium (IPDPS)*, 2019.

[4] Michael Hao Tong, Robert L. Grossman, and Haryadi S. Gunawi. Experiences in Managing the Performance and Reliability of a Large-Scale Genomics Cloud Platform. In *USENIX Annual Technical Conference (ATC)*, 2021.

[5] Sergei Yakneen, Sebastian M. Waszak, Michael Gertz, and Jan O. Korbel. Butler enables rapid cloud-based analysis of thousands of human genomes. *Nature Biotechnology*, 38(3):288–292, 2020.

[6] Alexander Dobin, Carrie A. Davis, Felix Schlesinger, Jorg Drenkow, Chris Zaleski, Sonali Jha, Philippe Batut, Mark Chaisson, and Thomas R. Gingeras. STAR: ultrafast universal RNA-seq aligner. *Bioinformatics*, 29(1):15–21, 2013.

[7] Azza E. Ahmed et al. Design considerations for workflow management systems use in production genomics research and the clinic. *Scientific Reports*, 11(1):21680, 2021.

[8] T. M. A. Do, Loïc Pottier, R. Ferreira da Silva, F. Suter, S. Caíno-Lores, M. Taufer, E. Deelman, and T. Peterka. Accelerating Scientific Workflows on HPC Platforms with In Situ Processing. In *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2022.

[9] Taylor Reiter, Phillip T Brooks, Luiz Irber, Shannon E K Joslin, Charles M Reid, Camille Scott, C Titus Brown, and N Tessa Pierce-Ward. Streamlining data-intensive biology with workflow systems. *GigaScience*, 10(1):giaa140, 2021.

[10] Anthony M. Bolger, Marc Lohse, and Bjoern Usadel. Trimmomatic: a flexible trimmer for Illumina sequence data. *Bioinformatics*, 30(15):2114–2120, 2014.

[11] Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: workload autoscaling at Google. In *European Conference on Computer Systems (EuroSys)*, 2020.

[12] Zibo Wang, Pinghe Li, Chieh-Jan Mike Liang, Feng Wu, and Francis Y. Yan. Autothrottle: A Practical Bi-Level Approach to Resource Management for SLO-Targeted Microservices. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024.

[13] Ataollah Fatahi Baarzi and George Kesidis. SHOWAR: Right-Sizing And Efficient Scheduling of Microservices. In *ACM Symposium on Cloud Computing (SoCC)*, 2021.

[14] Jiuchen Shi, Hang Zhang, Zhixin Tong, Quan Chen, Kaihua Fu, and Minyi Guo. Nodens: Enabling Resource Efficient and Fast QoS Recovery of Dynamic Microservice Applications in Datacenters. In *USENIX Annual Technical Conference (ATC)*, 2023.

[15] Yawen Wang, Kapil Arya, Marios Kogias, Manohar Vanga, Aditya Bhandari, Neeraja J. Yadwadkar, Siddhartha Sen, Sameh Elnikety, Christos Kozyrakis, and Ricardo Bianchini. SmartHarvest: Harvesting Idle CPUs Safely and Efficiently in the Cloud. In *European Conference on Computer Systems (EuroSys)*, 2021.

[16] Pradeep Ambati, Iñigo Goiri, Felipe Vieira Frujeri, Alper Gun, Ke Wang, Brian Dolan, Brian Corell, Sekhar Pasupuleti, Thomas Moscibroda, Sameh Elnikety, Marcus Fontoura, and Ricardo Bianchini. Providing SLOs for Resource-Harvesting VMs in Cloud Platforms. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

[17] Calin Iorgulescu, Reza Azimi, Youngjin Kwon, Sameh Elnikety, Manoj Syamala, Vivek R. Narasayya, Herodotos Herodotou, Paulo Tomita, Alex Chen, Jack Zhang, and Junhua Wang. PerfIso: Performance Isolation for Commercial Latency-Sensitive Services. In *USENIX Annual Technical Conference (ATC)*, 2018.

[18] Jonathan Bader, Joel Witzke, Soeren Becker, Ansgar Lößer, Fabian Lehmann, Leon Doehler, Anh Duc Vu, and Odej Kao. Towards Advanced Monitoring for Scientific Workflows. In *IEEE International Conference on Big Data (Big Data)*, 2022.

[19] Jonathan Bader, Lauritz Thamsen, Svetlana Kulagina, Jonathan Will, Henning Meyerhenke, and Odej Kao. Tarema: Adaptive Resource Allocation for Scalable Scientific Workflows in Heterogeneous Clusters. In *IEEE International Conference on Big Data (Big Data)*, 2021.

[20] Alok Singh, Shweta Purawat, Arvind Rao, and Ilkay Altintas. Modular performance prediction for scientific workflows using Machine Learning. *Future Generation Computer Systems*, 114:1–14, 2021.

[21] Ewa Deelman, Anirban Mandal, Ming Jiang, and Rizos Sakellariou. The role of machine learning in scientific workflows. *International Journal of High Performance Computing Applications*, 33(6), 2019.

[22] Overview of GDC Harmonization Workflows. https://github.com/NCI-GDC/gdc-workflow-overview.

[23] Heng Li and Richard Durbin. Fast and accurate short read alignment with Burrows–Wheeler transform. *Bioinformatics*, 25(14):1754–1760, 2009.

[24] German Tischler and Steven Leonard. biobambam: tools for read pair collation based algorithms on BAM files. *Source Code for Biology and Medicine*, 9:13, 2014.

[25] Oleksandr Rudyy, Marta Garcia-Gasulla, Filippo Mantovani, Alfonso Santiago, Raül Sirvent, and Mariano Vázquez. Containers in HPC: A Scalability and Portability Study in Production Biological Simulations. In *IEEE International Parallel & Distributed Processing Symposium (IPDPS)*, 2019.

[26] Stephen F. Altschul, Warren Gish, Webb Miller, Eugene W. Myers, and David J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215(3):403–410, 1990.

[27] Stepanka Zverinova and Victor Guryev. Variant calling: Considerations, practices, and developments. *Human Mutation*, 43(8):976–985, 2022.

[28] David C. Samuels, Hui Yu, and Yan Guo. Is it time to reassess variant annotation? *Trends in Genetics*, 38(6):521–523, 2022.

[29] Dong H. Ahn, Xiaohua Zhang, Jeffrey Mast, Stephen Herbein, Francesco Di Natale, Dan Kirshner, Sam Ade Jacobs, Ian Karlin, Daniel J. Milroy, Bronis R. de Supinski, Brian Van Essen, Jonathan E. Allen, and Felice C. Lightstone. Scalable Composition and Analysis Techniques for Massive Scientific Workflows. In *IEEE International Conference on eScience (eScience)*, 2022.

[30] Xiangqun Zheng-Bradley, Ian Streeter, Susan Fairley, David Richardson, Laura Clarke, Paul Flicek, and the 1000 Genomes Project Consortium. Alignment of 1000 Genomes Project reads to reference assembly GRCh38. *GigaScience*, 2017.

[31] Andy B. Yoo, Morris A. Jette, and Mark Grondona. SLURM: Simple Linux Utility for Resource Management. In *Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP)*, 2003.

[32] Robert L. Henderson. Job Scheduling Under the Portable Batch System. In *Workshop on Job Scheduling Strategies for Parallel Processing (JSSPP)*, 1995.

[33] Garrick Staples. TORQUE - TORQUE resource manager. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2006.

[34] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, Bikas Saha, Carlo Curino, Owen O'Malley, Sanjay Radia, Benjamin Reed, and Eric Baldeschwieler. Apache Hadoop YARN: yet another resource negotiator. In *ACM Symposium on Cloud Computing (SoCC)*, 2013.

[35] Johannes Weiner, Niket Agarwal, Dan Schatzberg, Leon Yang, Hao Wang, Blaise Sanouillet, Bikash Sharma, Tejun Heo, Mayank Jain, Chunqiang Tang, and Dimitrios Skarlatos. TMO: Transparent Memory Offloading in Datacenters. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2022.

[36] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Colleran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. Lessons Learned from the Chameleon Testbed. In *USENIX Annual Technical Conference (ATC)*, 2020.

[37] Henry Qin, Qian Li, Jacqueline Speiser, Peter Kraft, and John K. Ousterhout. Arachne: Core-Aware Thread Management. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[38] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2019.